

Este documento describe el contrato de configuración completo del API Gateway: todas las variables de entorno, archivos de configuración y esquemas de validación que el sistema necesita para funcionar correctamente.

Las siguientes variables se definen en `.env` o se inyectan como variables de entorno del sistema operativo o contenedor Docker. El archivo `.env.example` sirve como referencia canónica.

Variable	Categoría	Requerida / Default
Descripción		
PORT	Servidor	Opcional / 3000
Puerto TCP en el que el Gateway escucha conexiones entrantes.	HOST	Servidor
Opcional / 0.0.0.0	Dirección de enlace del servidor. 0.0.0.0 acepta conexiones externas.	CONFIG_PATH
Configuración	Opcional / ./config/gateway.yaml	Ruta al archivo YAML de configuración principal del Gateway.
REDIS_URL	Datos / Estado	Sí (Prod) (Default: redis://localhost:6379)
URL de conexión al servidor Redis (soporta redis:// y rediss://).	NODE_ENV	Runtime
Opcional / development	Entorno de ejecución (development, production o test).	LOG_LEVEL
Observabilidad	Opcional / info	Nivel mínimo de logs emitidos por Pino (debug, info, warn, error).

El Gateway se configura mediante un archivo YAML que se valida estrictamente con Zod en tiempo de carga. Soporta interpolación de variables de entorno con la sintaxis `\${VARIABLE}`.

Estructura del Esquema

El esquema raíz (`GatewayConfigSchema`) se compone de las siguientes secciones:

Sección `server`

Campo	Tipo	Default
Validación		
port	number	3000
Entero entre 1 y 65535	host	string
"0.0.0.0"	String no vacío	

Sección `redis`

Campo	Tipo	Default
Validación		
url	string	(requerido)
Debe comenzar con <code>redis://</code> o <code>rediss://</code>	onFailure	enum
"open"	\ "open\" o \ "closed\" . Comportamiento ante caída de Redis.	

Sección `logging`

Campo	Tipo	Default
Validación		
level	enum	"info"
\ "debug\" , \ "info\" , \ "warn\" , \ "error\"		

Sección `metrics`

Campo	Tipo	Default
Validación		
enabled	boolean	true
–	path	string
"/metrics"	Debe comenzar con \ /	defaultLabels
Record	{}	Mapa clave-valor de strings

Sección `routes` (array, mínimo 1 elemento)

Campo	Tipo	Default
Validación		
<code>prefix</code>	string	(requerido)
Debe comenzar con <code>\</code> /. No debe terminar con <code>\</code> /.	<code>target</code>	string
(requerido)	URL válida con protocolo <code>http://</code> o <code>https://</code> .	<code>stripPrefix</code>
boolean	false	—
<code>rateLimit</code>	object	(opcional)
Contiene <code>maxRequests</code> y <code>windowSeconds</code> (enteros positivos).	<code>timeout</code>	object
(opcional)	Contiene <code>connect</code> , <code>headers</code> , <code>body</code> (enteros ms).	<code>jwt</code>
object	(opcional)	Sub-esquema de autenticación JWT.
<code>metricsLabel</code>	string	(opcional)
Etiqueta personalizada para métricas de Prometheus.	<code>backendName</code>	string
(opcional)	Nombre del backend para identificación en logs.	<code>circuitBreaker</code>
object	(opcional)	Sub-esquema de Circuit Breaker.

Sub-esquema `circuitBreaker`

Campo	Tipo	Default
Validación		
<code>enabled</code>	boolean	true
—	<code>errorThreshold</code>	number
50	Entero entre 1 y 100 (porcentaje de error).	<code>requestCount</code>

Campo	Tipo	Default
Validación		
number	100	Entero positivo (ventana de evaluación).
recoveryTimeMs	number	30000
Entero positivo (milisegundos para half-open).	halfOpenRequests	number
3	Entero positivo (requests de prueba en half-open).	maxRetries
number	3	Entero no negativo.
retryDelayMs	number	100
Entero positivo (delay base del backoff).	retryMaxDelayMs	number
5000	Entero positivo (delay máximo del backoff).	

Sub-esquema `jwt`

Campo	Tipo	Default
Validación		
enabled	boolean	true
–	secret	string
(requerido)	String no vacío. Secreto para firmar/verificar tokens.	algorithm
enum	"HS256"	\ "HS256\" , \ "HS384\" , \ "HS512\"
forwardClaims	string[]	\ [\"sub\" , ...]
Array de strings no vacíos (claims inyectados al backend).		

El loader de configuración (`src/config/loader.ts`) soporta la interpolación de variables de entorno dentro del archivo YAML usando la sintaxis `\${NOMBRE_VARIABLE}` .

Ejemplo en `gateway.yaml` :

```
redis:
  url: \${REDIS_URL}
```

Si la variable referenciada no existe en el entorno, el sistema lanza una excepción tipada `MissingEnvVarError` e impide el arranque.

El `ConfigReloader` permite modificar la configuración sin reiniciar el proceso, enviando la señal `SIGHUP` al proceso del Gateway.

Campos recargables en caliente (se aplican de inmediato)

- `logging.level` : El nivel de log se actualiza al instante.
- `routes\[n\].rateLimit` : Los límites de tasa por ruta se aplican al siguiente request.
- `overrides` : Los overrides de Rate Limiting se recargan atómicamente.

Campos NO recargables (se ignoran, requieren reinicio)

- `server.port` y `server.host` : El socket TCP ya está enlazado.
- `redis.url` y `redis.onFailure` : La conexión a Redis ya está establecida.
- `routes\[n\].prefix` y `routes\[n\].target` : Las rutas están registradas estáticamente en Fastify.
- `routes\[n\].timeout` : Los timeouts se configuran en el binding de Undici al crear el pool.